



BCSL305 - Lab Manual-2 - yes

Digital Design and Computer Organization (Vidya Vikas Institute of Engineering and Technology)



Scan to open on Studocu



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DATA STRUCTURES AND APPLICATION
LAB MANUAL

BCSL305

Namithadevi N. N.

INDEX

SL NO.	CONTENT	PAGE NO.
1.	<u>Program 01: Calendar Application</u>	01-03
2.	<u>Program 02: String Operations</u>	04-06
3.	<u>Program 03: Stack of Integers</u>	07-11
4.	<u>Program 04: Infix to Postfix Conversion</u>	12-14
5.	<u>Program 05: Stack Applications</u> a. <u>Evaluation of Suffix expression</u> b. <u>Tower of Hanoi Problem</u>	15-19
6.	<u>Program 06: Circular Queue</u>	20-23
7.	<u>Program 07: Singly Linked List of Student Data</u>	24-29
8.	<u>Program 08: Doubly Linked List of Employee Data</u>	30-35
9.	<u>Program 09: Polynomial Evaluation and Addition</u>	36-40
10.	<u>Program 10: Binary Search Tree</u>	41-46
11.	<u>Program 11: Graph Reachability using BFS/DFS</u> a. <u>BFS Method</u> b. <u>DFS Method</u>	47-50
12.	<u>Program 12: Hashing & Linear Probing</u>	51-54

LAB 01

Develop a Program in C for the following:

- Declare a calendar as an array of 7 elements (A dynamically Created array) to represent 7 days of a week. Each Element of the array is a structure having three fields. The first field is the name of the Day (A dynamically allocated String), The second field is the date of the Day (A integer), the third field is the description of the activity for a particular day (A dynamically allocated String).
- Write functions create(), read() and display(); to create the calendar, to read the data from the keyboard and to print weeks activity details report on screen.

```
#include <stdio.h>
#include <stdlib.h>

struct Day{
    char *dayName;
    int date;
    int month;
    int year;
    char *activity;
};

void create(struct Day* day){
    day-> dayName=(char *)malloc(sizeof(char)*20);
    day-> activity=(char *)malloc(sizeof(char)*100);
    printf("Enter the day name: ");
    scanf("%s", day->dayName);
    printf("Enter the date in dd mm yyyy format: ");
    scanf("%d %d %d", &day->date, &day->month, &day->year);
    printf("Enter the activity for the day: ");
    fflush(stdin);
    fgets(day->activity, 100, stdin);
}

void read(struct Day *calender, int size){
    for(int i=0; i<size; i++){
        printf("\nEnter the details for day %d: \n", i+1);
        create(&calender[i]);
    }
}

void display(struct Day *calender, int size){
    printf("\nWeek's activity Details: \n");
    for(int i=0; i<size; i++){
        printf("Day: %d\t\v", i+1);
        printf("DayName: %s\t", -9, calender[i].dayName);
        printf("Date: %d/%d/%d\t", calender[i].date, calender[i].month, calender[i].year);
    }
}
```

```

        printf("Activity: %s",calender[i].activity);
        printf("\n");
    }
}

void freemem(struct Day *calender, int size){
    for(int i=0; i<size; i++){
        free(calender[i].dayName);
        free(calender[i].activity);
    }
}

int main(){
    int size;
    printf("Enter the number of days in the week: ");
    scanf("%d", &size);
    struct Day *calender=(struct Day*)malloc(sizeof(struct Day)*size);
    if(calender==NULL){
        printf("Memory allocation failed, Exiting.....");
        return 1;
    }

    read(calender,size);
    display(calender,size);
    freemem(calender,size);

    free(calender);

    return 0;
}

```

OUTPUT:

```

Enter the number of days in the week: 7

Enter the details for day 1:
Enter the day name: Monday
Enter the date in dd mm yyyy format: 01 01 2024
Enter the activity for the day: DSA Lab

Enter the details for day 2:
Enter the day name: Tuesday
Enter the date in dd mm yyyy format: 02 01 2024
Enter the activity for the day: Java Lab

Enter the details for day 3:
Enter the day name: Wednesday
Enter the date in dd mm yyyy format: 03 01 2024
Enter the activity for the day: DDCO Lab

```

Enter the details for day 4:
Enter the day name: Thursday
Enter the date in dd mm yyyy format: 04 01 2024
Enter the activity for the day: OS Lab

Enter the details for day 5:
Enter the day name: Friday
Enter the date in dd mm yyyy format: 05 01 2024
Enter the activity for the day: Excel Lab

Enter the details for day 6:
Enter the day name: Saturday
Enter the date in dd mm yyyy format: 06 01 2024
Enter the activity for the day: Holiday

Enter the details for day 7:
Enter the day name: Sunday
Enter the date in dd mm yyyy format: 07 01 2024
Enter the activity for the day: Holiday

Week's activity Details:

Day: 1

DayName: Monday Date: 1/1/2024 Activity: DSA Lab

Day: 2

DayName: Tuesday Date: 2/1/2024 Activity: Java Lab

Day: 3

DayName: Wednesday Date: 3/1/2024 Activity: DDCO Lab

Day: 4

DayName: Thursday Date: 4/1/2024 Activity: OS Lab

Day: 5

DayName: Friday Date: 5/1/2024 Activity: Excel Lab

Day: 6

DayName: Saturday Date: 6/1/2024 Activity: Holiday

Day: 7

DayName: Sunday Date: 7/1/2024 Activity: Holiday

Run:



LAB 02

Develop a Program in C for the following operations on Strings.

- a) Read a main String (STR), a Pattern String (PAT) and a Replace String (REP)
- b) Perform Pattern Matching Operation: Find and Replace all occurrences of PAT in STR with REP if PAT exists in STR. Report suitable messages in case PAT does not exist in STR.

Support the program with functions for each of the above operations. Don't use Built-in functions.

Background:

Strings are actually one-dimensional array of characters terminated by a null character `'\0'`. Thus, a null-terminated string contains the characters that comprise the string followed by a null.

The following declaration and initialization create a string consisting of the word `"Hello"`.

To hold the null character at the end of the array, the size of the character array containing the string is one more than the number of characters in the word `"Hello"`.

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

If you follow the rule of array initialization then you can write the above statement as follows: `char greeting[] = "Hello";`

C language supports a wide range of built-in functions that manipulate null-terminated strings as follows:

```
strcpy(s1, s2); //Copies string s2 into string s1.
strcat(s1, s2); //Concatenates string s2 onto the end of string s1.
strlen(s1); //Returns the length of string s1.
strcmp(s1, s2); //Returns 0 if s1 and s2 are the same; less than 0 if s1 is
lexicographically smaller than s2.
strchr(s1, ch); //Returns a pointer to the first occurrence of character ch in string
s1.
strstr(s1, s2); //Returns a pointer to the first occurrence of string s2 in string
s1.
```

Algorithm:

- Step 1: Start.
- Step 2: Read main string STR, pattern string PAT and replace string REP.
- Step 3: Search/find the pattern string PAT in the main string STR.
- Step 4: If PAT is found then replace all occurrences of PAT in main string STR with REP string.
- Step 5: If PAT is not found give a suitable error message.
- Step 6: Stop.

```

#include <stdio.h>

int main(){

    char STR[100], PAT[100], REP[100], ans[100];
    int i, j, c, m, k, flag = 0; // Declare variables for indexing and a flag to check if
    pattern is found

    printf("\nEnter the main string: \n"); fgets(STR, 100, stdin);

    printf("\nEnter the pattern string: \n"); fgets(PAT, 100, stdin);

    printf("\nEnter the replace string: \n"); fgets(REP, 100, stdin);

    // Initialize the indices (i, j, c, m) for pattern and string traversal
    i = m = c = j = 0;

    // Start a loop to traverse through the main string (STR)
    while(STR[c] != '\0'){ // Continue until the end of the string is reached
        // Check if the current character of STR matches the current character of PAT
        if(STR[m] == PAT[i]){
            i++; // Move to the next character of PAT
            m++; // Move to the next character of STR
            flag = 1; // Set flag to 1 indicating that we found a match
            // Check if we have reached the end of the pattern string (PAT)
            if(PAT[i] == '\0'){
                // If the pattern has been fully matched, replace it with REP
                for(k = 0; REP[k] != '\0'; k++, j++) // Copy characters from REP to ans
                    ans[j] = REP[k];
                i = 0; // Reset the pattern index i to start again
                c = m; // Move the current index c to the next position in STR after the
                matched pattern
            }
        }
        else{
            // If no match, copy the current character of STR to ans
            ans[j] = STR[c];
            j++; // Increment the result string index
            c++; // Increment the index of the main string STR
            m = c; // Set m to the current index in STR (for next matching)
            i = 0; // Reset the pattern index to start searching again
        }
    }

    // After completing the loop, check if a pattern was found and replaced
    if(flag == 0){
        printf("Pattern not found"); // If flag is still 0, no pattern was matched
    }
    else{
        ans[j] = '\0'; // Null-terminate the resultant string
    }
}

```

```
// Print the final result string with replacements
printf("\nThe resultant string is: %s\n", ans);
}

return 0;
}
```

OUTPUT:

```
Enter the main string:
Good Morning

Enter the pattern string:
Morning

Enter the replace string:
Evening

The resultant string is: Good Evening
```

Run:

LAB 03

Develop a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX)

- Push an Element on to Stack
- Pop an Element from Stack
- Demonstrate how Stack can be used to check Palindrome
- Demonstrate Overflow and Underflow situations on Stack
- Display the status of Stack
- Exit.

Support the program with appropriate functions for each of the above operations.

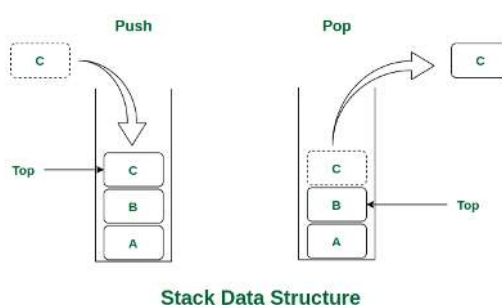
Background:

A stack is an abstract data type (ADT), commonly used in most programming languages. It is named stack as it behaves like a real-world stack. A real-world stack allows operations at one end only. For example, we can place or remove a card or plate from top of the stack only.

Likewise, Stack ADT allows all data operations at one end only. At any given time, we can only access the top element of a stack. This feature makes it LIFO data structure. *LIFO stands for Last-in-first-out*. Here, the element which is placed (inserted or added) last is accessed first. In stack terminology, insertion operation is called *PUSH* operation and removal operation is called *POP* operation.

A stack can be implemented by means of Array, Structure, Pointer and Linked-List. Stack can either be a fixed size one or it may have a sense of dynamic resizing.

Here, we are going to implement stack using arrays which makes it a fixed size stack implementation.



Basic Operations:

- `push()`: pushing (storing) an element on the stack.
- `pop()`: removing (accessing) an element from the stack.

To use a stack efficiently we need to check status of stack as well. For the same purpose, the following functionality is added to stacks:

- `peek()`: get the top data element of the stack, without removing it.
- `isFull()`: check if stack is full.
- `isEmpty()`: check if stack is empty.

Algorithm:

Step 1: Start.

Step 2: Initialize stack size to MAX and top of stack to -1.

Step 3: Push integer element on to stack and display the contents of the stack. If stack is full print a message as "Stack Overflow".

Step 4: Pop element from stack and display the contents of the stack. If stack is empty print a message as "Stack Underflow".

Step 5: Check whether the contents of the stack are a Palindrome or not.

Step 6: Stop.

```
#include <stdio.h>
#include <stdlib.h>

#define max 10
int stack[max], top=-1, flag=1;
int i, temp, item, rev[max], num[max];

void push(){
    if(top==(max-1)){
        printf("Stack overflow");
    }
    else{

        printf("Enter the elements to be inserted: ");
        scanf("%d", &item);
        top=top+1;
        stack[top]=item;
    }
}

void pop(){
    if(top==-1){
        printf("\nStack underflow\n");
        flag=0;
    }
    else{

        item=stack[top];
        top=top-1;
    }
}

void palindrome(){
    i = 0;
    if(top == -1){
        printf("\nPush some elements into the stack first\n");
    }
    else{
```

```

// Copy the elements of the stack to a temporary array 'rev' in reverse order
// This will allow us to compare the original stack order with the reversed order
while(top != -1){
    rev[top] = stack[top]; // Copy the element from stack to 'rev' array
    pop();
}

// Save the original position of the top pointer to restore it later
top = temp;

// Compare the elements of the stack with the reversed 'rev' array
for(i = 0; i <= temp; i++){
    // Check if the current element of the stack matches the corresponding element
    in 'rev'
    if(stack[top--] == rev[i]){
        if(i == temp){
            printf("Palindrome\n");
            return;
        }
    }
}
printf("Not a Palindrome\n");
}
}

void display(){
    int i;
    top=temp;
    if(top== -1){
        printf("\nStack is empty");
    }
    else{
        printf("\nThe elements of the stack are: \n");
        for(i=top; i>=0; i--){
            printf("%d\n", stack[i]);
        }
    }
}

int main(){
    int choice;
    printf("\n-----Stack Operations-----\n");
    printf("1.Push 2.Pop 3.Palindrome 4.Display 5.Exit\n");
    while(1){
        temp=top;
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        switch(choice){
            case 1: push();
                    break;
            case 2: pop();

```

```

        if(flag)
            printf("\nPopped element: %d\t", item);
            temp=top;
            break;
    case 3: palindrome();
            top=temp;
            break;
    case 4: display();
            break;
    case 5: exit(0);
            break;
    default:
            printf("\nInvalid choice\n");
            break;
    }
}
return 0;
}

```

OUTPUT:

-----Stack Operations-----
 1.Push 2.Pop 3.Palindrome 4.Display 5.Exit

Enter your choice: 1
 Enter the elements to be inserted: 10

Enter your choice: 1
 Enter the elements to be inserted: 20

Enter your choice: 1
 Enter the elements to be inserted: 30

Enter your choice: 1
 Enter the elements to be inserted: 20

Enter your choice: 1
 Enter the elements to be inserted: 10

Enter your choice: 1
 Enter the elements to be inserted: 60

Enter your choice: 4

The elements of the stack are:

60
 10
 20
 30

```
20
10

Enter your choice: 2

Popped element: 60
Enter your choice: 4

The elements of the stack are:
10
20
30
20
10

Enter your choice: 3
Palindrome

Enter your choice: 5
```

Run:



LAB 04

Develop a Program in C for converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: +, -, *, /, % (Remainder), ^ (Power) and alphanumeric operands.

Background:

Infix: Operators are written in-between their operands. Ex: $X + Y$

Prefix: Operators are written before their operands. Ex: $+X Y$

Postfix: Operators are written after their operands. Ex: $XY+$

Examples of Infix, Prefix, and Postfix

Infix Expression	Prefix Expression	Postfix Expression
$A + B$	$+ A B$	$A B +$
$A + B * C$	$+ A * B C$	$ABC*+$

Expression	Stack	Output	Comment
$5^E + D * (C^B + A)$	Empty	-	Initial
$^E + D * (C^B + A)$	Empty	5	Print
$E + D * (C^B + A)$	^	5	Push
$+ D * (C^B + A)$	^	5E	Push
$D * (C^B + A)$	+	5E^	Pop and Push
$*(C^B + A)$	+	5E^D	Print
$(C^B + A)$	+(	5E^D	Push
$C^B + A)$	+(	5E^D	Push
$^B + A)$	+(	5E^DC	Print
$B + A)$	+(	5E^DC	Push
$+ A)$	+(	5E^DCB	Print
$A)$	+(	5E^DCB^	Pop and Push
$)$	+(	5E^DCB^A	Print
End	+	5E^DCB^A+	Pop until '('
End	Empty	5E^DCB^A+*	Pop every element

Infix to prefix conversion Expression = $(A+B^C)*D+E^5$

Step 1. Reverse the infix expression. $5^E + D * (C^B + A)$

Step 2. Make Every '(' as ')' and every ')' as '('. $5^E + D * (C^B + A)$

Step 3. Convert expression to postfix form.

Step 4. Reverse the expression. $+*+A^BCD^E$

Step 5. Result. $+*+A^BCD^E5$

Algorithm:

- Step 1: Start.
 Step 2: Read an infix expression with parenthesis and without parenthesis.
 Step 3: Convert the infix expression to postfix expression.
 Step 4: Stop.

```

#define SIZE 50
#include <ctype.h>
#include <stdio.h>

char s[SIZE];
int top = -1;

void push(char elem)
{
    s[++top] = elem;
}

char pop()
{
    return (s[top--]);
}

int pr(char elem)
{
    switch (elem)
    {
        case '#':
            return 0;
        case '(':
            return 1;
        case '+':
        case '-':
            return 2;
        case '*':
        case '/':
        case '%':
            return 3;
        case '^':
            return 4;
    }
}

int main()
{
    char infx[50], pofx[50], ch, elem;
    int i = 0, k = 0;

    printf("\nRead the infix expresssion: ");

```

```

scanf("%s", infx);
push('#');
while ((ch = infx[i++]) != '\0')
{
    if (ch == '(')
    {
        push(ch);
    }
    else if (isalnum(ch))
    {
        pofx[k++] = ch;
    }
    else if (ch == ')')
    {
        while (s[top] != '(')
            pofx[k++] = pop();
        elem = pop();
    }
    else
    {
        while (pr(s[top]) >= pr(ch))
            pofx[k++] = pop();
        push(ch);
    }
}
while (s[top] != '#')
    pofx[k++] = pop();
pofx[k] = '\0';
printf("\nGiven Infix expression: %s \nPostfix expression: %s\n\n", infx, pofx);

return 0;
}

```

OUTPUT:

Read the infix expresssion: (a+b)*c/d^5%1

Given Infix expression: (a+b)*c/d^5%1

Postfix expression: ab+c*d5^/1%

Run:



LAB 05

Develop a Program in C for the following Stack Applications:

- Evaluation of Suffix expression with single digit operands and operators: +, -, *, /, %, ^
- Solving Tower of Hanoi problem with n disks

```
//Lab5a.c

#include <stdio.h>
#include <math.h>

#define MAX 20
char postfix[MAX];

struct stack {
    int top;
    float str[MAX];
} s;

int isoperand(char x) {
    if(x >= '0' && x <= '9')
        return 1;
    else
        return 0;
}

void push(float x) {
    if(s.top == MAX - 1) {
        printf("Stack is full\nStack overflow\n");
        return;
    } else {
        s.top++;
        s.str[s.top] = x;
    }
}

float pop() {
    if(s.top == -1) {
        printf("Stack is empty\nStack Underflow\n");
        return -1;
    } else {
        s.top--;
        return s.str[s.top + 1];
    }
}

float operate(float op1, float op2, char a) {
    switch(a) {
        case '+':
            return op2 + op1;
```

```

    case '-' :
        return op2 - op1;
    case '*' :
        return op2 * op1;
    case '/' :
        return op2 / op1;
    case '%':
        return (int)op2 % (int)op1;
    case '^' :
        return pow(op2, op1);
    default:
        printf("Error: Invalid operator %c\n", a);
        return 0;
    }
}

int main() {
    int i = 0;
    printf("Enter the expression:");
    scanf("%s", postfix);
    float ans, op1, op2;
    while(postfix[i] != '\0') {
        if(isoperand(postfix[i]))
            push(postfix[i] - 48);
        else {
            op1 = pop();
            op2 = pop();
            ans = operate(op1, op2, postfix[i]);
            push(ans);
            printf("%.2f %c %.2f = %.2f\n", op2, postfix[i], op1, ans);
        }
        i++;
    }
    printf("Result: %.2f", s.str[s.top]);

    return 0;
}

```

OUTPUT 1:

```

Enter the expression:
123*+4-
2.00 * 3.00 =6.00
1.00 + 6.00 =7.00
7.00 - 4.00 =3.00
Result: 3.00

```

OUTPUT 2:

```

Enter the expression:
22^32*+
2.00 ^ 2.00 =4.00
3.00 * 2.00 =6.00
4.00 + 6.00 =10.00
Result: 10.00

```

OUTPUT 3:

```

Enter the expression:52%
5.00 % 2.00 = 1.00
Result: 1.00

```

Run:



To change directories in the Linux terminal, you use the `cd` (change directory) command. Here's how it works:

```
cd /path/to/directory
```

The `-lm` flag is used to link the math library when compiling a C program with `gcc`.

Compile: `gcc test.c -o test.o -lm` Run: `./test.o`

In C, mathematical functions like `sin()`, `cos()`, `sqrt()`, `log()`, and many others are part of the math library (`libm`). These functions are not included in the standard C library (`libc`) by default, so when you use them, you need to explicitly link against the math library.

```
hj2024@kali: /media/hj2024/6480578C80576418/Users/HP/Downloads/Local Send/DSA Lab
File Actions Edit View Help

(hj2024@kali)-[/media/.../HP/Downloads/Local Send/DSA Lab]
$ gcc Lab5a.c -o Lab5a.o -lm

(hj2024@kali)-[/media/.../HP/Downloads/Local Send/DSA Lab]
$ ./Lab5a.o
Enter the expression: 123*+4-
2.00 * 3.00 =6.00
1.00 + 6.00 =7.00
7.00 - 4.00 =3.00
Result: 3.00

(hj2024@kali)-[/media/.../HP/Downloads/Local Send/DSA Lab]
$
```

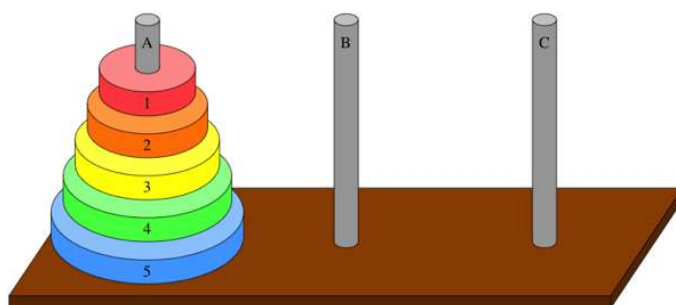
Background:

Solving Tower of Hanoi problem with n disks. The Tower of Hanoi is a mathematical game or puzzle. It consists of three rods, and a number of disks of different sizes which can slide onto any rod. The puzzle starts with the disks in a neat stack in ascending order of size on one rod, the smallest at the top, thus making a conical shape.

The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules:

1. Only one disk can be moved at a time.
2. Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack.
3. No disk may be placed on top of a smaller disk. With three disks, the puzzle can be solved in seven moves.

The minimum number of moves required to solve a Tower of Hanoi puzzle is $2^n - 1$, where n is the number of disks.



```
//Lab5b.c
```

```
#include <stdio.h>
#include <math.h>

void tower(int n, int source, int temp, int destination){
    if(n==0)
        return;
    tower(n-1, source, destination, temp);
    printf("\nMove disk %d from %c to %c", n, source, destination);
    tower(n-1, temp, source, destination);
}

void main(){
    int n;
    printf("\nEnter the number of disks: ");
    scanf("%d", &n);
    tower(n, 'A', 'B', 'C');
```

```
printf("\n\nTotal number of moves are: %d\n", (int)pow(2,n)-1);
}
```

OUTPUT:

Enter the number of disks: 3

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Total number of moves are: 7

Run:



```

hj2024@kali: /media/hj2024/6480578C80576418/Users/HP/Downloads/Local Send/DSA Lab
File Actions Edit View Help

(hj2024@kali)-[/media/.../HP/Downloads/Local Send/DSA Lab]
$ gcc Lab5b.c -o Lab5b.o -lm

(hj2024@kali)-[/media/.../HP/Downloads/Local Send/DSA Lab]
$ ./Lab5b.o

Enter the number of disks: 3

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Total number of moves are: 7

(hj2024@kali)-[/media/.../HP/Downloads/Local Send/DSA Lab]
$ 

```

LAB 06

Develop a menu driven Program in C for the following operations on Circular QUEUE of Characters (Array Implementation of Queue with maximum size MAX)

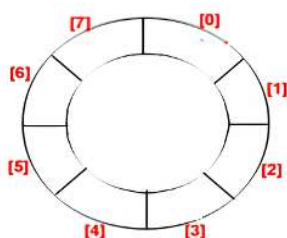
- Insert an Element on to Circular QUEUE
- Delete an Element from Circular QUEUE
- Demonstrate Overflow and Underflow situations on Circular QUEUE
- Display the status of Circular QUEUE
- Exit

Support the program with appropriate functions for each of the above operations.

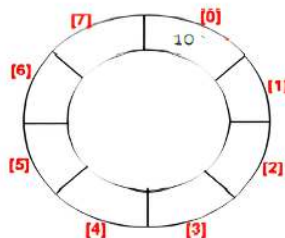
Background:

Circular queue is a linear data structure. It follows FIFO principle. In circular queue the last node is connected back to the first node to make a circle. Circular linked list follows the First-In-First-Out principle. Elements are added at the rear end and the elements are deleted at front end of the queue. The queue is considered as a circular queue when the positions 0 and MAX 1 are adjacent. Any position before front is also after rear.

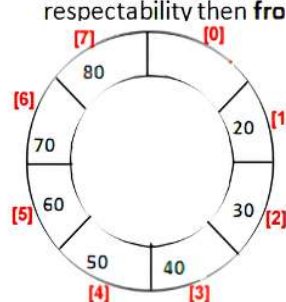
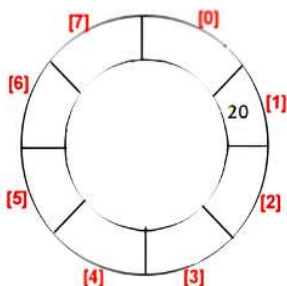
1) Initially: **Front = 0** and **rear = -1**



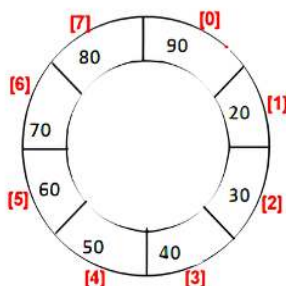
2) Add item 10 then **front = 0** and **rear = 0**.



3) Now delete one item then **front = 1** and **rear = 1**. 4) Like this now insert 30, 40, and 50, 50, 70, 80 respectively then **front = 1** and **rear = 7**.



5) Now in case of linear queue, we can not access 0 block for insertion but in circular queue next item will be inserted of 0 block then **front = 0** and **rear = 0**.



```
//Lab6.c

#include <stdio.h>
#include <stdlib.h>
#define SIZE 5

int CQ[SIZE];
int front = -1;
int rear = -1, ch;

void CQ_Insert(int item) {
    if(front == -1)
        front++;
    rear = (rear + 1) % SIZE;
    CQ[rear] = item;
}

void CQ_Delete() {
    int item;
    item = CQ[front];
    printf("Deleted element is: %d\n", item);
    front = (front + 1) % SIZE;
}

void CQ_Display() {
    int i;
    if(front == -1)
        printf("Circular Queue is Empty.\n");
    else {
        printf("Elements of the circular queue are: \n");
        for(i = front; i != rear; i = (i + 1) % SIZE)
        {
            printf("%d\t", CQ[i]);
        }
        printf("%d\n", CQ[i]);
    }
}

int IsCQ_Full() {
    if(front == (rear + 1) % SIZE)
        return 1;
    return 0;
}

int IsCQ_Empty() {
    if(front == -1)
        return 1;
    else if(front == rear) {
        printf("Deleted element is: %d", CQ[front]);
        front = -1;
        return 1;
    }
}
```

```

    }
    return 0;
}

void main() {
    printf("1.Insert\n2.Delete\n3.Display\n4.Exit\n");
    while(1) {
        int ele;
        printf("Enter your choice\n");
        scanf("%d", &ch);
        switch(ch) {
            case 1:
                if(IsCQ_Full())
                    printf("Circular Queue Overflow\n");
                else {
                    printf("Enter the element to be inserted: ");
                    scanf("%d", &ele);
                    CQ_Insert(ele);
                }
                break;
            case 2:
                if(IsCQ_Empty())
                    printf("Circular Queue Underflow\n");
                else
                    CQ_Delete();
                break;
            case 3:
                if(IsCQ_Empty())
                    printf("Circular Queue Underflow\n");
                else
                    CQ_Display();
                break;
            case 4:
                exit(0);

            default:
                printf("Invalid Choice.\n");
                break;
        }
    }
}

```

OUTPUT:

```

1.Insert
2.Delete
3.Display
4.Exit
Enter your choice

```

```
1
Enter the element to be inserted: 10
Enter your choice
1
Enter the element to be inserted: 20
Enter your choice
1
Enter the element to be inserted: 30
Enter your choice
1
Enter the element to be inserted: 40
Enter your choice
1
Enter the element to be inserted: 50
Enter your choice
1
Circular Queue Overflow
Enter your choice
3
Elements of the circular queue are:
10      20      30      40      50
Enter your choice
2
Deleted element is: 10
Enter your choice
3
Elements of the circular queue are:
20      30      40      50
Enter your choice
4
```

Run:



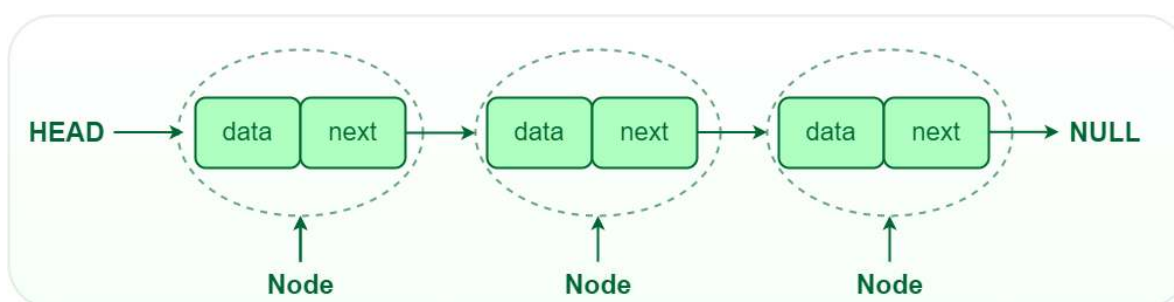
LAB 07

Develop a menu driven Program in C for the following operations on Singly Linked List (SLL) of Student Data with the fields: *USN, Name, Programme, Sem, PhNo*.

- Create a SLL of N Students Data by using *front insertion*.
- Display the status of SLL and count the number of nodes in it
- Perform Insertion / Deletion at End of SLL
- Perform Insertion / Deletion at Front of SLL (Demonstration of stack)
- Exit

Background:

Linked List is a linear data structure and it is very common data structure which consists of group of nodes in a sequence which is divided in two parts. Each node consists of its own data and the address of the next node and forms a chain. Linked Lists are used to create trees and graphs.



- They are a dynamic in nature which allocates the memory when required.
- Insertion and deletion operations can be easily implemented.
- Stacks and queues can be easily executed.
- Linked List reduces the access time.
- Linked lists are used to implement stacks, queues, graphs, etc.
- Linked lists let you insert elements at the beginning and end of the list.
- In Linked Lists we don't need to know the size in advance.

Singly Linked List: Singly linked lists contain nodes which have a data part as well as an address part i.e. next, which points to the next node in sequence of nodes. The operations we can perform on singly linked lists are insertion, deletion and traversal.

Algorithm:

- Step 1: Start.
- Step 2: Read the value of N. (N student's information)
- Step 3: Create a singly linked list. (SLL)
- Step 4: Display the status of SLL.
- Step 5: Count the number of nodes.

- Step 6: Perform insertion at front of list.
 Step 7: Perform deletion at the front of the list.
 Step 8: Perform insertion at end of the list.
 Step 9: Perform deletion at the end of the list.
 Step 10: Demonstrate how singly linked list can be used as stack.
 Step 11: Demonstrate how singly linked list can be used as queue.
 Step 12: Stop.

```
//Lab7.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct{
    int usn;
    char name[20];
    char branch[20];
    int semester;
    char phone[20];
}student;

struct node{
    int usn;
    char name[20];
    char branch[20];
    int semester;
    char phone[20];
    struct node *link;
};

typedef struct node *Node;

Node getnode(){
    Node x;
    x=(Node)malloc(sizeof(struct node));
    if(x==NULL){
        printf("Out of memory\n");
        exit(0);
    }
    return x;
}

Node insert_front(student item, Node first){
    Node temp;
    temp=getnode();
    temp->usn=item.usn;
    strcpy(temp->name, item.name);
    strcpy(temp->branch, item.branch);
    temp->semester=item.semester;
    strcpy(temp->phone, item.phone);
```

```

temp->link=NULL;
if(first==NULL)
    return temp;
temp->link=first;
return temp;
}

Node insert_rear(student item, Node first){
    Node temp, cur;
    temp=getnode();
    temp->usn=item.usn;
    strcpy(temp->name, item.name);
    strcpy(temp->branch, item.branch);
    temp->semester=item.semester;
    strcpy(temp->phone, item.phone);
    temp->link=NULL;
    if(first==NULL)
        return temp;
    cur=first;
    while(cur->link!=NULL){
        cur=cur->link;
    }
    cur->link=temp;
    return first;
}

Node delete_front(Node first){
    Node temp;
    if(first==NULL){
        printf("Student list is empty\n");
        return NULL;
    }
    temp=first;
    temp=temp->link;
    printf("Delete student record: USN=%d\n", first->usn);
    free(first);
    return temp;
}

Node delete_rear(Node first){
    Node cur, prev;
    if(first==NULL){
        printf("Student list is empty, cannot delete\n");
        return first;
    }
    if(first->link==NULL){
        printf("Delete student record: USN=%d\n", first->usn);
        free(first);
        return NULL;
    }
    prev=NULL;

```

```

    cur=first;
    while(cur->link!=NULL){
        prev=cur;
        cur=cur->link;
    }
    printf("Delete student record: USN=%d\n", cur->usn);
    free(cur);
    prev->link=NULL;
    return first;
}

void display(Node first){
    Node cur;
    int count=0;
    if(first==NULL){
        printf("Student list is empty\n");
        return;
    }
    cur=first;
    while(cur!=NULL){
        printf("%d\t%s\t%s\t%d\t%s\t\n", cur->usn, cur->name, cur->branch, cur->semester,
cur->phone);
        cur=cur->link;
        count++;
    }
    printf("Number of students= %d\n", count);
}

void main(){
    Node first;
    int choice;
    student item;
    first=NULL;
    for(;;){
        printf("\n1. Insert_front 2.Insert_rear 3.Delete_front 4.Delete_rear 5.Display
6.Exit\n");
        printf("Enter the choice: ");
        scanf("%d", &choice);
        switch(choice){
            case 1: printf("\nUSN: ");
                    scanf("%d", &item.usn);
                    printf("Name: ");
                    scanf("%s", item.name);
                    printf("Branch: ");
                    scanf("%s", item.branch);
                    printf("Semester: ");
                    scanf("%d", &item.semester);
                    printf("Phone: ");
                    scanf("%s", item.phone);
                    first=insert_front(item, first);
                    break;

```

```

        case 2: printf("\nUSN: ");
                scanf("%d", &item.usn);
                printf("Name: ");
                scanf("%s", item.name);
                printf("Branch: ");
                scanf("%s", item.branch);
                printf("Semester: ");
                scanf("%d", &item.semester);
                printf("Phone: ");
                scanf("%s", item.phone);
                first=insert_rear(item, first);
                break;
        case 3: first=delete_front(first);
                break;
        case 4: first=delete_rear(first);
                break;
        case 5: display(first);
                break;
        default: exit(0);
    }
}
}

```

OUTPUT:

1. Insert_front 2.Insert_rear 3.Delete_front 4.Delete_rear 5.Display 6.Exit
Enter the choice: 1

USN: 01
Name: HJ
Branch: CSE
Semester: 3
Phone: 9632587410

1. Insert_front 2.Insert_rear 3.Delete_front 4.Delete_rear 5.Display 6.Exit
Enter the choice: 2

USN: 02
Name: PJ
Branch: ECE
Semester: 3
Phone: 8523697410

1. Insert_front 2.Insert_rear 3.Delete_front 4.Delete_rear 5.Display 6.Exit
Enter the choice: 5

1	HJ	CSE	3	9632587410
2	PJ	ECE	3	8523697410

NUmber of students= 2

1. Insert_front 2.Insert_rear 3.Delete_front 4.Delete_rear 5.Display 6.Exit

Enter the choice: 3

Delete student record: USN=1

1. Insert_front 2.Insert_rear 3.Delete_front 4.Delete_rear 5.Display 6.Exit

Enter the choice: 5

2 PJ ECE 3 8523697410

Number of students= 1

1. Insert_front 2.Insert_rear 3.Delete_front 4.Delete_rear 5.Display 6.Exit

Enter the choice: 4

Delete student record: USN=2

1. Insert_front 2.Insert_rear 3.Delete_front 4.Delete_rear 5.Display 6.Exit

Enter the choice: 5

Student list is empty

1. Insert_front 2.Insert_rear 3.Delete_front 4.Delete_rear 5.Display 6.Exit

Enter the choice: 6

Run:



LAB 08

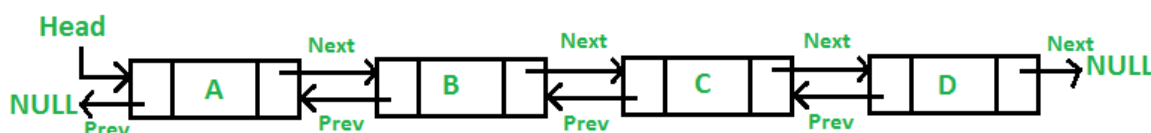
Develop a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: *SSN, Name, Dept, Designation, Sal, PhNo*

- Create a DLL of N Employees Data by using *end insertion*.
- Display the status of DLL and count the number of nodes in it
- Perform Insertion and Deletion at End of DLL
- Perform Insertion and Deletion at Front of DLL
- Demonstrate how this DLL can be used as Double Ended Queue.
- Exit

Background:

Doubly Linked List: In a doubly linked list, each node contains two links the first link points to the previous node and the next link points to the next node in the sequence. In computer science, a doubly linked list is a linked data structure that consists of a set of sequentially linked records called nodes. Each node contains two fields, called links, that are references to the previous and to the next node in the sequence of nodes.

The beginning and ending nodes' previous and next links, respectively, point to some kind of terminator, typically a sentinel node or null, to facilitate traversal of the list. If there is only one sentinel node, then the list is circularly linked via the sentinel node. It can be conceptualized as two singly linked lists formed from the same data items, but in opposite sequential orders. A doubly linked list whose nodes contain three fields: an integer value, the link to the next node, and the link to the previous node.



The two-node links allow traversal of the list in either direction. While adding or removing a node in a doubly linked list requires changing more links than the same operations on a singly linked list, the operations are simpler and potentially more efficient (for nodes other than first nodes) because there is no need to keep track of the previous node during traversal or no need to traverse the list to find the previous node, so that its link can be modified.

Algorithm:

- Step 1: Start.
- Step 2: Read the value of N. (N student's information)
- Step 3: Create a doubly linked list (DLL)

- Step 4: Display the status of DLL.
 Step 5: Count the number of nodes.
 Step 6: Perform insertion at front of list.
 Step 7: Perform deletion at the front of the list.
 Step 8: Perform insertion at end of the list.
 Step 9: Perform deletion at the end of the list.
 Step 10: Demonstrate how doubly linked list can be used as double ended queue.
 Step 11: Stop.

```
//Lab8.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Encode {
    char ssn[12];
    char name[20];
    char dept[10];
    char desig[10];
    int salary;
    long long int phno;
    struct Encode *left;
    struct Encode *right;
} *head = NULL;

struct Encode *tail, *temp1, *temp2;

void create(char s[12], char n[20], char dpt[10], char des[10], int sal, long long int p)
{
    if(head == NULL) {
        head = (struct Encode *)malloc(sizeof(struct Encode));
        strcpy(head->:ssn, s);
        strcpy(head->name, n);
        strcpy(head->dept, dpt);
        strcpy(head->desig, des);
        head->salary = sal;
        head->phno = p;
        head->left = NULL;
        head->right = NULL;
        tail = head;
    } else {
        temp1 = (struct Encode *)malloc(sizeof(struct Encode));
        strcpy(temp1->:ssn, s);
        strcpy(temp1->name, n);
        strcpy(temp1->dept, dpt);
        strcpy(temp1->desig, des);
        temp1->salary = sal;
        temp1->phno = p;
        tail->right = temp1;
        temp1->right = NULL;
    }
}
```

```

        temp1->left = tail;
        tail = temp1;
    }
}

void display() {
    temp1 = head;
    printf("Employee details\n");
    while(temp1 != NULL) {
        printf("\n");
        printf("%s\n%s\n%s\n%s\n%d\n%lld\n", temp1->:ssn, temp1->name, temp1->dept, temp1->desig, temp1->salary, temp1->phno);
        temp1 = temp1->right;
    }
}

void ins_beg(char s[12], char n[20], char dpt[10], char des[10], int sal, long long int p)
{
    temp1 = (struct Encode *)malloc(sizeof(struct Encode));
    strcpy(temp1->:ssn, s);
    strcpy(temp1->name, n);
    strcpy(temp1->dept, dpt);
    strcpy(temp1->desig, des);
    temp1->salary = sal;
    temp1->phno = p;
    temp1->right = head;
    head->left = temp1;
    head = temp1;
    temp1->left = NULL;
}

void ins_end(char s[12], char n[20], char dpt[10], char des[10], int sal, long long int p)
{
    temp1 = (struct Encode *)malloc(sizeof(struct Encode));
    strcpy(temp1->:ssn, s);
    strcpy(temp1->name, n);
    strcpy(temp1->dept, dpt);
    strcpy(temp1->desig, des);
    temp1->salary = sal;
    temp1->phno = p;
    temp1->right = NULL;
    temp1->left = tail;
    tail->right = temp1;
    tail = temp1;
}

void delete_beg() {
    if(head == NULL) {
        printf("List is empty\n");
        return;
    }
}

```

```

temp1 = head->right;
free(head);
if(temp1 == NULL) {
    head = tail = NULL; // List is now empty
} else {
    head = temp1;
    head->left = NULL;
}
}

void delete_end() {
    if(tail == NULL) {
        printf("List is empty\n");
        return;
    }
    temp1 = tail->left;
    free(tail);
    if(temp1 == NULL) {
        head = tail = NULL; // List is now empty
    } else {
        tail = temp1;
        tail->right = NULL;
    }
}

int main() {
    int choice;
    char s[12], n[20], dpt[10], des[10];
    int sal;
    long long int p;

    printf("1. Create\n2. Display\n3. Insert at beginning\n4. Insert at end\n5. Delete at
beginning\n6. Delete at end\n7. Exit\n");
    while(1) {
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                printf("Enter the required data (Emp No, Name, Dept, Desig, Salary, PhNo)\n");
                scanf("%s%s%s%s%d%lld", s, n, dpt, des, &sal, &p);
                create(s, n, dpt, des, sal, p);
                break;
            case 2:
                display();
                break;
            case 3:
                printf("Enter the required data (Emp No, Name, Dept, Desig, Salary, PhNo)\n");
                scanf("%s%s%s%s%d%lld", s, n, dpt, des, &sal, &p);
                ins_beg(s, n, dpt, des, sal, p);
                break;
            case 4:

```

```

        printf("Enter the required data (Emp No, Name, Dept, Desig, Salary, PhNo)\n");
        scanf("%s%s%s%s%d%lld", s, n, dpt, des, &sal, &p);
        ins_end(s, n, dpt, des, sal, p);
        break;
    case 5:
        delete_beg();
        break;
    case 6:
        delete_end();
        break;
    case 7:
        exit(0);
    }
}
}

```

OUTPUT:

1. Create
2. Display
3. Insert at beginning
4. Insert at end
5. Delete at beginning
6. Delete at end
7. Exit

Enter your choice: 1
 Enter the required data (Emp No, Name, Dept, Desig, Salary, PhNo)
 01 HJ Sales Manager 50000 9632587410

Enter your choice: 4
 Enter the required data (Emp No, Name, Dept, Desig, Salary, PhNo)
 02 PJ Product Manager 60000 8523697410

Enter your choice: 3
 Enter the required data (Emp No, Name, Dept, Desig, Salary, PhNo)
 00 KJ Design Head 80000 7412589630

Enter your choice: 2
 Employee details

00
 KJ
 Design
 Head
 80000
 7412589630

01

HJ
Sales
Manager
50000
9632587410

02
PJ
Product
Manager
60000
8523697410

Enter your choice: 5

Enter your choice: 2
Employee details

01
HJ
Sales
Manager
50000
9632587410

02
PJ
Product
Manager
60000
8523697410

Enter your choice: 6

Enter your choice: 2
Employee details

01
HJ
Sales
Manager
50000
9632587410

Enter your choice: 7

Run:



LAB 09

Develop a Program in C for the following operations on Singly Circular Linked List (SCLL) with header nodes

- Represent and Evaluate a Polynomial $P(x,y,z) = 6x^2y^2z - 4yz^5 + 3x^3yz + 2xy^5z - 2xyz^3$
- Find the sum of two polynomials $POLY1(x,y,z)$ and $POLY2(x,y,z)$ and store the result in $POLYSUM(x,y,z)$

Support the program with appropriate functions for each of the above operations.

Background:

Circular Linked List: In the circular linked list, the last node of the list contains the address of the first node and forms a circular chain.

A **polynomial equation** is a mathematical expression composed of variables, coefficients, and exponents. It can be written in the general form:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0, \text{ where:}$$

- $a_n, a_{n-1}, \dots, a_0$ are constants (coefficients of the terms),
- x is the variable,
- n is a non-negative integer (the degree of the polynomial).

When $P(x) = 0$, this equation becomes a polynomial equation.

Here are the steps required for addition of two polynomials:

- Arrange the Polynomial in standard form
- Standard form of a polynomial and each of the following terms just means that the term with highest degree is first
- Arrange the like terms in columns and add the like terms

Example: Let's find the sum of the following two polynomials:

$$(3y^5 + y^4 + 2y^3 - 2y + 5) \text{ and } (2y^5 + 3y^3 + 7y + 2)$$

1) Write in Standard form $(3y^5 + y^4 + 2y^3 - 2y + 5) + (2y^5 + 3y^3 + 7y + 2)$

2) Arrange in columns of like terms and then add

$$\begin{array}{r}
 3y^5 + y^4 + 2y^3 - 2y + 5 \\
 2y^5 \quad \quad + 3y^3 + 7y + 2 \\
 \hline
 5y^5 + y^4 + 5y^3 + 5y + 7
 \end{array}$$

Algorithm:

- Step 1: Start.
 Step 2: Read a polynomial.
 Step 3: Represent the polynomial using singly circular linked list.
 Step 4: Evaluate the given polynomial
 Step 5: Read two polynomials and find the sum of the polynomials.
 Step 6: Stop

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Node structure for a term in the polynomial
typedef struct Node {
    int coeff; // Coefficient
    int x, y, z; // Powers of x, y, z
    struct Node *next; // Pointer to the next node
} Node;

// Function to create a new node
Node* createNode(int coeff, int x, int y, int z) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->coeff = coeff;
    newNode->x = x;
    newNode->y = y;
    newNode->z = z;
    newNode->next = newNode; // Points to itself (circular list)
    return newNode;
}

// Function to insert a term into the polynomial
void insertTerm(Node *header, int coeff, int x, int y, int z) {
    Node *newNode = createNode(coeff, x, y, z);
    Node *temp = header;

    // Traverse to the end of the circular list
    while(temp->next != header) {
        temp = temp->next;
    }
    // Insert the new node
    temp->next = newNode;
    newNode->next = header;
}

// Function to display the polynomial
void displayPoly(Node *header) {
    if(header->next == header) {
        printf("0\n");
        return;
    }
    Node *temp = header->next;
```

```

while(temp != header) {
    printf("%d(x^%d)(y^%d)(z^%d) ", temp->coeff, temp->x, temp->y, temp->z);
    temp = temp->next;
}
printf("\n");
}

// Function to evaluate the polynomial
int evaluatePoly(Node *header, int xVal, int yVal, int zVal) {
    int result = 0;
    Node *temp = header->next;
    while(temp != header) {
        result += temp->coeff * pow(xVal, temp->x) * pow(yVal, temp->y) * pow(zVal, temp->z);
        temp = temp->next;
    }
    return result;
}

// Function to add two polynomials
Node* addPolynomials(Node *poly1, Node *poly2) {
    Node *result = createNode(0, 0, 0, 0); // Header node for result
    Node *p1 = poly1->next;
    Node *p2 = poly2->next;
    while(p1 != poly1 || p2 != poly2) {
        if(p1 != poly1 && (p2 == poly2 || (p1->x == p2->x && p1->y == p2->y && p1->z == p2->z))) {
            insertTerm(result, p1->coeff + (p2 != poly2 ? p2->coeff : 0), p1->x, p1->y, p1->z);
            p1 = p1->next;
            if(p2 != poly2) p2 = p2->next;
        } else if(p1 != poly1) {
            insertTerm(result, p1->coeff, p1->x, p1->y, p1->z);
            p1 = p1->next;
        } else {
            insertTerm(result, p2->coeff, p2->x, p2->y, p2->z);
            p2 = p2->next;
        }
    }
    return result;
}

int main() {
    // Polynomial P(x, y, z)
    Node *poly1 = createNode(0, 0, 0, 0); // Header node
    insertTerm(poly1, 6, 2, 2, 1);
    insertTerm(poly1, -4, 0, 1, 5);
    insertTerm(poly1, 3, 3, 1, 1);
    insertTerm(poly1, 2, 1, 5, 1);
    insertTerm(poly1, -2, 1, 1, 3);

    printf("Polynomial P(x, y, z):\n");
}

```

```

displayPoly(poly1);

// Polynomial Q(x, y, z)
Node *poly2 = createNode(0, 0, 0, 0); // Header node
insertTerm(poly2, 2, 2, 2, 1);
insertTerm(poly2, 1, 0, 1, 5);
insertTerm(poly2, 4, 3, 1, 1);

printf("\nPolynomial Q(x, y, z):\n");
displayPoly(poly2);

// Sum of P and Q
Node *polySum = addPolynomials(poly1, poly2);
printf("\nSum of P(x, y, z) and Q(x, y, z):\n");
displayPoly(polySum);

// Evaluate P(x, y, z) at x=1, y=2, z=3
int result = evaluatePoly(poly1, 1, 2, 1);
printf("\nValue of P(x, y, z) at x=1, y=2, z=1: %d\n", result);

return 0;
}

```

OUTPUT:

```

(hj2024@kali) - [/media/.../HP/Downloads/Local Send/DSA Lab]
$ gcc SCLL9.c -o SCLL9.o -lm

(hj2024@kali) - [/media/.../HP/Downloads/Local Send/DSA Lab]
$ ./SCLL9.o
Polynomial P(x, y, z):
+6(x^2)(y^2)(z^1) -4(x^0)(y^1)(z^5) +3(x^3)(y^1)(z^1) +2(x^1)(y^5)(z^1) -2(x^1)(y^1)(z^3)

Polynomial Q(x, y, z):
+2(x^2)(y^2)(z^1) +1(x^0)(y^1)(z^5) +4(x^3)(y^1)(z^1)

Sum of P(x, y, z) and Q(x, y, z):
+8(x^2)(y^2)(z^1) -3(x^0)(y^1)(z^5) +7(x^3)(y^1)(z^1) +2(x^1)(y^5)(z^1) -2(x^1)(y^1)(z^3)

Value of P(x, y, z) at x=1, y=2, z=1: 82

(hj2024@kali) - [/media/.../HP/Downloads/Local Send/DSA Lab]
$

```

Polynomial $P(x, y, z)$:

$$+6(x^2)(y^2)(z^1) - 4(x^0)(y^1)(z^5) + 3(x^3)(y^1)(z^1) + 2(x^1)(y^5)(z^1) - 2(x^1)(y^1)(z^3)$$

Polynomial $Q(x, y, z)$:

$$+2(x^2)(y^2)(z^1) + 1(x^0)(y^1)(z^5) + 4(x^3)(y^1)(z^1)$$

Sum of $P(x, y, z)$ and $Q(x, y, z)$:

$$+8(x^2)(y^2)(z^1) - 3(x^0)(y^1)(z^5) + 7(x^3)(y^1)(z^1) + 2(x^1)(y^5)(z^1) - 2(x^1)(y^1)(z^3)$$

Value of $P(x, y, z)$ at $x=1, y=2, z=1$: 82

Run:



LAB 10

Develop a menu driven Program in C for the following operations on Binary Search Tree (BST) of Integers.

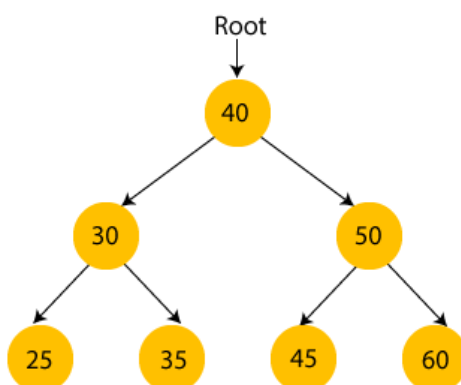
- Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2
- Traverse the BST in Inorder, Preorder and Post Order
- Search the BST for a given element (KEY) and report the appropriate message
- Exit

Background:

A binary search tree (BST) is a tree in which all nodes follow the below mentioned properties-

- The left sub-tree of a node has key less than or equal to its parent node's key.
- The right sub-tree of a node has key greater than or equal to its parent node's key.

Thus, a binary search tree (BST) divides all its sub-trees into two segments; left sub-tree and right sub-tree and can be defined as $left_subtree(key) \leq node(key) \leq right_subtree(key)$



Following are basic primary operations of a tree:

- Search – search an element in a tree.
- Insert – insert an element in a tree.
- Preorder Traversal – traverse a tree in a preorder manner.
- Inorder Traversal – traverse a tree in an inorder manner.
- Postorder Traversal – traverse a tree in a postorder manner.

Node definition: Define a node having some data, references to its left and right child nodes.

```

struct node {
    int data;
    struct node *leftChild;
    struct node *rightChild;
};
  
```

Algorithm:

- Step 1: Start.
- Step 2: Create a Binary Search Tree for N elements.
- Step 3: Traverse the tree in inorder.
- Step 4: Traverse the tree in preorder.
- Step 5: Traverse the tree in postorder.
- Step 6: Search the given key element in the BST.
- Step 7: Delete an element from BST.
- Step 8: Stop

```
#include <stdio.h>
#include <stdlib.h>

struct BST {
    int data;
    struct BST *left;
    struct BST *right;
};

typedef struct BST NODE;

// Function to create a new node in the BST
NODE* createtree(NODE* node, int data) {
    if(node == NULL) {
        NODE *temp = (NODE *)malloc(sizeof(NODE));
        temp->data = data;
        temp->left = temp->right = NULL;
        return temp;
    }
    if(data < node->data) {
        node->left = createtree(node->left, data);
    } else if(data > node->data) {
        node->right = createtree(node->right, data);
    }
    return node;
}

// Function to search for an element in the BST
NODE* search(NODE* node, int data) {
    if(node == NULL) {
        printf("\nElement not found\n");
        return NULL;
    } else if(data < node->data) {
        return search(node->left, data);
    } else if(data > node->data) {
        return search(node->right, data);
    } else {
        printf("\nElement found: %d\n", node->data);
        return node;
    }
}
```

```

}

// Inorder traversal
void inorder(NODE* node) {
    if(node != NULL) {
        inorder(node->left);
        printf("%d\t", node->data);
        inorder(node->right);
    }
}

// Preorder traversal
void preorder(NODE* node) {
    if(node != NULL) {
        printf("%d\t", node->data);
        preorder(node->left);
        preorder(node->right);
    }
}

// Postorder traversal
void postorder(NODE* node) {
    if(node != NULL) {
        postorder(node->left);
        postorder(node->right);
        printf("%d\t", node->data);
    }
}

int main() {
    int data, choice, i, n, key;
    NODE *root = NULL;

    while(1) {
        printf("\n\n1. Insertion in BST");
        printf("\n2. Search element in BST");
        printf("\n3. Inorder traversal");
        printf("\n4. Preorder traversal");
        printf("\n5. Postorder traversal");
        printf("\n6. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("\nEnter number of elements to insert in BST: ");
                scanf("%d", &n);
                printf("\nEnter the values to create BST:\n");
                for(i = 0; i < n; i++) {
                    scanf("%d", &data);
                    root = createtree(root, data);
                }
            }
        }
    }
}

```

```

    }
    break;

case 2:
    printf("\nEnter the element to search: ");
    scanf("%d", &key);
    search(root, key);
    break;

case 3:
    printf("\nInorder traversal:\n");
    inorder(root);
    break;

case 4:
    printf("\nPreorder traversal:\n");
    preorder(root);
    break;

case 5:
    printf("\nPostorder traversal:\n");
    postorder(root);
    break;

case 6:
    exit(0);

default:
    printf("\nInvalid choice\n");
    break;
}
}
}

```

OUTPUT:

```

1. Insertion in BST
2. Search element in BST
3. Inorder traversal
4. Preorder traversal
5. Postorder traversal
6. Exit

```

Enter your choice: 1

Enter number of elements to insert in BST: 12

Enter the values to create BST:

6 9 5 2 8 15 24 14 7 8 5 2

1. Insertion in BST
2. Search element in BST
3. Inorder traversal
4. Preorder traversal
5. Postorder traversal
6. Exit

Enter your choice: 2

Enter the element to search: 15

Element found: 15

1. Insertion in BST
2. Search element in BST
3. Inorder traversal
4. Preorder traversal
5. Postorder traversal
6. Exit

Enter your choice: 3

Inorder traversal:

2 5 6 7 8 9 14 15 24

1. Insertion in BST
2. Search element in BST
3. Inorder traversal
4. Preorder traversal
5. Postorder traversal
6. Exit

Enter your choice: 4

Preorder traversal:

6 5 2 9 8 7 15 14 24

1. Insertion in BST
2. Search element in BST
3. Inorder traversal
4. Preorder traversal
5. Postorder traversal
6. Exit

Enter your choice: 5

Postorder traversal:

2 5 7 8 14 24 15 9 6

1. Insertion in BST
2. Search element in BST
3. Inorder traversal
4. Preorder traversal
5. Postorder traversal

6. Exit

Enter your choice: 6

Run:



LAB 11

Develop a Program in C for the following operations on Graph(G) of Cities

- Create a Graph of N cities using Adjacency Matrix.
- Print all the nodes reachable from a given starting node in a digraph using DFS/BFS method

Background:

Adjacency Matrix

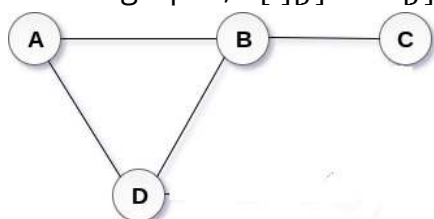
In graph theory and computer science, an **adjacency matrix** is a square matrix used to represent a finite graph. The elements of the matrix indicate whether pairs of vertices are adjacent or not in the graph.

For a graph $G = (V, E)$, where $V = \{0, 1, 2, \dots, n - 1\}$ represents the set of vertices, the graph can be represented using a two-dimensional integer array of size $n \times n$. For example, a matrix $a[20][20]$ can store a graph with 20 vertices.

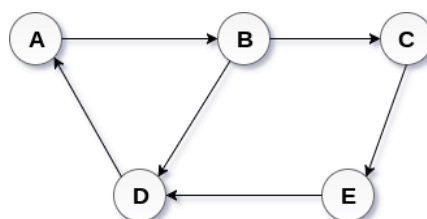
- If $a[i][j] = 1$, it indicates the presence of an edge between vertices i and j .
- If $a[i][j] = 0$, it indicates the absence of an edge between vertices i and j .

Properties of the Adjacency Matrix:

- Undirected Graph:** The adjacency matrix of an undirected graph is always symmetric. This means that an edge (i, j) implies the edge (j, i) . Thus, for undirected graphs, $a[i][j] = a[j][i]$.



Undirected Graph



Directed Graph

- Directed Graph:** The adjacency matrix of a directed graph is **not symmetric**. For a directed edge (i, j) , the matrix entry $a[i][j] = 1$ indicates a directed edge from vertex i to vertex j , but the reverse, $a[j][i]$, is not necessarily 1.

Breadth-First Search (BFS)

Breadth-First Search (BFS) is an algorithm for traversing or searching a graph or tree. It starts at the root node (or any arbitrary node in the case of a graph) and explores the neighbouring nodes before moving to the next level of neighbours.

BFS uses a **queue** to remember the next vertex to explore, which ensures that all vertices at the current level are explored before moving to the next level.

BFS Algorithm:

1. **Visit the adjacent unvisited vertex:** Mark it as visited, display it, and insert it into the queue.
2. **If no adjacent vertex is found:** Remove the first vertex from the queue.
3. **Repeat steps 1 and 2:** Continue until the queue is empty.

Depth-First Search (DFS)

Depth-First Search (DFS) is an algorithm for traversing or searching a graph. It starts at an arbitrary vertex and explores as far as possible along each branch before backtracking. Unlike BFS, which explores all neighbouring vertices at the current level before moving to the next level, DFS explores deeply into the graph before visiting other branches.

In DFS, a **stack** is used to keep track of the vertices to explore. The algorithm explores as deeply as possible, pushing vertices onto the stack and popping them off when backtracking is required.

DFS Algorithm:

1. **Push the starting vertex** onto the stack and mark it as visited.
2. **While the stack is not empty**, do the following:
 - Pop a vertex from the stack and process it.
 - For each unvisited adjacent vertex of the popped vertex, push it onto the stack and mark it as visited.
3. **Repeat** step 2 until the stack is empty.

In DFS, each vertex has three possible states:

- **White** (unvisited)
- **Gray** (visited but with unvisited neighbours)
- **Black** (fully explored)

Algorithm:

- Step 1: Start.
- Step 2: Input the value of N nodes of the graph
- Step 3: Create a graph of N nodes using adjacency matrix representation.
- Step 4: Print the nodes reachable from the starting node using BFS.
- Step 5: Check whether graph is connected or not using DFS.
- Step 6: Stop.

```

#include <stdio.h>
#include <stdlib.h>

int a[20][20], q[20], visited[20], reach[10];
int n, i, j, f=0, r=-1, count=0;

void bfs(int v){
    for(i=1; i<=n; i++)
        if(a[v][i] && !visited[i])
            q[++r]=i;
    if(f<=r){
        visited[q[f]]=1;
        bfs(q[f++]);
    }
}

void dfs(int v){
    int i; reach[v]=1;
    for(i=0; i<=n; i++){
        if(a[v][i] && !reach[i]){
            printf("\n %d->%d", v, i);
            count++;
            dfs(i);
        }
    }
}

int main(){
    int v, choice;
    printf("\nEnter the number of vertices: ");
    scanf("%d", &n);
    for(i=1; i<=n; i++){
        q[i]=0;
        visited[i]=0;
    }
    for(i=1; i<=n-1; i++)
        reach[i]=0;
    printf("\nEnter graph data in matrix form: \n");
    for(i=1; i<=n; i++){
        for(j=1; j<=n; j++){
            scanf("%d", &a[i][j]);
        }
    }
    printf("1.BFS 2.DFS 3.EXIT\n");
    printf("\nEnter your choice: ");
    scanf("%d", &choice);
    switch(choice){
        case 1: printf("\nEnter the starting vertex: ");
                scanf("%d", &v);
                bfs(v);
                if((v<1)|| (v>n)){

```

```

        printf("\nBFS is not possible\n");
    }
    else{
        printf("\nThe nodes which are reachable from %d: \n", v);
        for(i=1; i<=n; i++)
            if(visited[i])
                printf("%d\t", i);
        }
        break;
    case 2: dfs(1);
        if(count==n-1)
            printf("\nGraph is connected\n");
        else
            printf("\nGraph is not connected\n");
        break;
    case 3: exit(0);
        break;
    default:
        printf("\nInvalid choice\n");
    }
}

```

OUTPUT 1:

Enter the number of vertices: 5

Enter graph data in matrix form:

0 1 0 1 0

1 0 1 0 1

0 1 0 1 0

1 0 1 0 1

0 1 0 1 0

1.BFS 2.DFS 3.EXIT

Enter your choice: 1

Enter the starting vertex: 1

The nodes which are reachable from 1:

1 2 3 4 5

OUTPUT 2:

Enter the number of vertices: 5

Enter graph data in matrix form:

0 1 0 1 0

1 0 1 0 1

0 1 0 1 0

1 0 1 0 0

0 1 0 0 0

1.BFS 2.DFS 3.EXIT

Enter your choice: 2

1->2

2->3

3->4

2->5

Run:

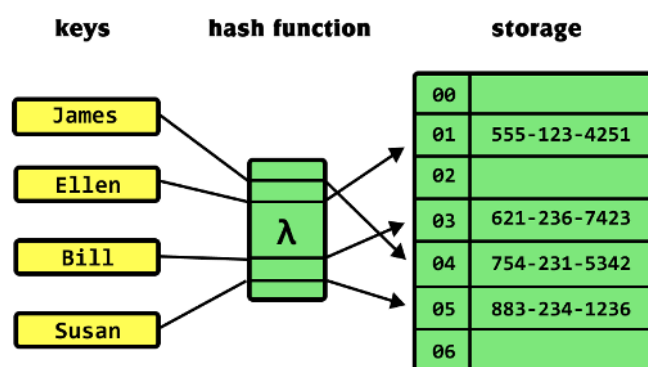


LAB 12

Given a File of 'N' employee records with a set 'K' of Keys (4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table (HT) of 'm' memory locations with 'L' as the set of memory addresses (2-digit) of locations in HT. Let the keys in 'K' and addresses in 'L' are Integers.

Develop a Program in C that uses Hash function $H: K \rightarrow L$ as $H(K) = K \bmod m$ (remainder method), and implement hashing technique to map a given key 'K' to the address space 'L'. Resolve the collision (if any) using linear probing.

Background:



Collision Handling Using Linear Probing

Hashing is a technique used to store and retrieve data efficiently. It uses a hash function to map keys to indices in a hash table. However, multiple keys may generate the same hash value, leading to a collision. Collision handling is critical to ensure the integrity of the hash table.

One popular method for resolving collisions is linear probing. In linear probing:

1. If a collision occurs (i.e., the hashed index is already occupied), the algorithm searches for the next available slot in the hash table by incrementing the index sequentially.
2. The search wraps around to the beginning of the table using the modulo operation if it reaches the end.

This program demonstrates collision handling using linear probing. It:

1. Accepts numerical inputs and maps them to indices using a simple hash function:
 $key = num \% 100.$
2. Handles collisions by probing subsequent slots until an empty one is found.
3. Offers two display options:
 - Show the entire hash table.
 - Show only the slots containing data.

This approach ensures efficient utilization of the hash table while maintaining quick access to stored data.

Algorithm:

Step 1: Start.

Step 2: Given a File of 'N' employee records with a set 'K' of Keys (4-digit) which uniquely determine the records in file 'F'.

Step 3: Assume that file 'F' is maintained in memory by a Hash Table (HT) of 'm' memory locations with 'L' as the set of memory addresses (2-digit) of locations in HT.

Step 4: Let the keys in K and addresses in L are Integers

Step 5: Hash function $H:K \rightarrow L$ as $H(K) = K \bmod m$ (remainder method)

Step 6: Hashing as to map a given key 'K' to the address space 'L', resolve the collision (if any) is using linear probing.

Step 7: Stop.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int create(int);
void display(int []);
void linear_prob(int [], int, int);

int main() {
    int a[MAX], num, key, i;
    int ans = 1;
    printf("Collision handling by linear probing: \n");
    for(i = 0; i < MAX; i++) {
        a[i] = -1;
    }
    do {
        printf("\nEnter the data: ");
        scanf("%4d", &num);
        key = create(num);
        linear_prob(a, key, num);
        printf("\nDo you wish to continue? (1/0): ");
        scanf("%d", &ans);
    } while(ans);

    display(a);
}

int create(int num) {
    int key;
    key = num % 100; // Hash function: key is the remainder when divided by 100
    return key;
}

void linear_prob(int a[MAX], int key, int num) {
    int i;
```

```

// If no collision, place the number at the hashed index
if (a[key] == -1) {
    a[key] = num;
    return;
}

printf("\nCollision detected...\n");

// If the position is already occupied, we perform linear probing
i = key;
while (a[i] != -1) { // Continue probing until an empty spot is found
    i = (i + 1) % MAX; // Wrap around using modulo if we go past the end
}

a[i] = num; // Insert the number in the empty spot
printf("Collision avoided successfully using linear probing.\n");
}

void display(int a[MAX]) {
    int i, choice;
    printf("1. Display all\n2. Filtered data\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    if(choice == 1) {
        printf("\nThe hash table is: \n");
        for(i = 0; i < MAX; i++) {
            printf("%.2d -> %d\n", i, a[i]);
        }
    }
    else {
        printf("\nThe filtered hash table is: \n");
        for(i = 0; i < MAX; i++) {
            if(a[i] != -1) {
                printf("%.2d -> %d\n", i, a[i]);
            }
        }
    }
}
}

```

OUTPUT:

Collision handling by linear probing:

Enter the data: 1234

Do you wish to continue? (1/0): 1

Enter the data: 2548

```
Do you wish to continue? (1/0): 1

Enter the data: 3256

Do you wish to continue? (1/0): 1

Enter the data: 1299

Do you wish to continue? (1/0): 1

Enter the data: 1298

Do you wish to continue? (1/0): 1

Enter the data: 1398

Collision detected...
Collision avoided successfully using linear probing.

Do you wish to continue? (1/0): 0
1. Display all
2. Filtered data
Enter your choice: 2

The filtered hash table is:
00 -> 1398
34 -> 1234
48 -> 2548
56 -> 3256
98 -> 1298
99 -> 1299
```

Run:

